OneSensor: An ESP32-Based Intelligent Virtual Multi-Sensor Platform for Cost-Effective IoT Development and Testing

Mahesh Avhad
Department of Computer Engineering
Pimpri Chinchwad College of Engineering Pune
City, India
mahesh.avhad24@pccoepune.org

Suyog Jadhav
Department of Computer Engineering
Pimpri Chinchwad College of Engineering Pune
City, India
suyog.jadhav24@pccoepune.org

Parshwa Nikam
Department of Computer Engineering
Pimpri Chinchwad College of Engineering Pune
City, India
parshwa.nikam24@pccoepune.org

Abstract – IoT development and testing often require multiple physical sensors, which increases hardware cost, wiring complexity, configuration effort, and maintenance requirements. Testing an IoT application under different environmental conditions or sensor failure situations is also difficult because such conditions are not always safe or easy to reproduce. This paper presents OneSensor, an ESP32-based virtual multi-sensor platform designed to provide configurable sensor inputs for IoT development and testing. The proposed system uses an ESP32 as a virtual sensor device capable of generating values representing sensors such as temperature, humidity, gas, light, and soil moisture, and the current prototype extends this set to twelve concurrently addressable channels. These values can be provided to an external device such as an Arduino UNO, which acts as the system under test. A web-based dashboard allows the user to configure virtual sensors, assign sensors to output channels, select environmental scenarios, and introduce controlled sensor faults. The proposed approach enables repeatable testing without requiring a separate physical sensor for every development stage. The platform also provides a foundation for future machine-learning-based sensor models, protocol-level sensor emulation, and automated testing. OneSensor therefore aims to provide a low-cost and reusable platform for controlled IoT experimentation and Hardware-in-the-Loop testing.

Index Terms – fault injection, IoT testing, sensor emulation, sensor virtualization, virtual sensors, hardware-in-the-loop testing

1. Introduction

The Internet of Things has become an important technology for applications including smart homes, agriculture, industrial monitoring, environmental monitoring, healthcare, and automation. These applications depend heavily on sensor data to observe physical conditions and make decisions. During the development of such systems, developers generally require several physical sensors to reproduce the inputs expected by the application. This increases the overall cost of prototyping and introduces additional wiring, calibration, power, and maintenance requirements.

Another important challenge is testing an IoT application under different environmental and abnormal conditions. A developer may need to determine how an application behaves when temperature increases rapidly, gas concentration rises, soil moisture decreases, communication is interrupted, or a sensor produces incorrect values. Reproducing these conditions using physical sensors can be difficult, unsafe, and inconsistent. Furthermore, repeatedly testing the same failure condition may require manually changing the physical environment or disconnecting hardware components.

Virtual sensing and sensor virtualization have been investigated to reduce the dependence on direct physical sensing resources. Yuriyama and Kushida proposed a Sensor-Cloud Infrastructure in which physical sensors could be represented and managed through virtual sensor objects [1]. Perera et al. introduced CA4IOT to provide context-aware configuration and processing of sensor information according to user requirements [2]. Their work demonstrates how higher-level context can be used to simplify interaction with large numbers of sensors.

Perera et al. later proposed a Mobile Sensor Hub that supported dynamic configuration of heterogeneous sensors and communication between sensor devices and middleware [3]. Bose and Mukherjee proposed SensIaaS, extending the concept of sensor virtualization into a sensor-cloud infrastructure [4]. Chatterjee et al. developed a Big-Sensor-Cloud Infrastructure for provisioning sensing resources as a service [5]. More recently, Zaidan et al. investigated virtual sensors for high-resolution air-pollution monitoring using artificial intelligence and IoT [6]. Complementary to the cloud-oriented virtual-sensor literature, a separate body of work on Hardware-in-the-Loop (HIL) simulation has studied how to electrically emulate sensors and actuators so that an embedded controller under test perceives a physically realistic signal rather than a purely software value [7, 8], and a further line of research studies how faults can be deliberately injected into such emulated signals or into the target system itself to evaluate reliability and fail-safe behavior [9, 10, 11].

Although these studies establish important concepts related to virtual sensors, context awareness, dynamic configuration, and intelligent sensing, the proposed OneSensor system focuses on a different application. Instead of primarily managing physical sensors through cloud infrastructure or estimating an unavailable physical measurement, OneSensor uses an embedded ESP32 device to generate controllable virtual sensor inputs for another IoT device. The ESP32 acts as the virtual sensing platform, while an Arduino UNO or another controller can act as the system under test.

The proposed platform combines virtual sensor generation, configurable sensor-to-channel mapping, environmental scenario simulation, fault injection, and browser-based control. This combination allows developers to test IoT applications under controlled conditions without requiring all corresponding physical sensors during the early stages of development. Compared with the initial concept, the current prototype additionally realizes the virtual-sensor-to-target link as a genuine electrical signal – pulse-width-modulated (PWM) pulses and an on-chip digital-to-analog converter (DAC) voltage – carried over physical wires to the target microcontroller's input pins, which places OneSensor firmly within the HIL tradition of electrical sensor emulation [7, 8] rather than a purely software-level mock.

2. Related Work

Sensor virtualization provides an abstraction between sensing resources and applications. Yuriyama and Kushida introduced a Sensor-Cloud Infrastructure that represents physical sensing resources through virtual sensors and virtual sensor groups [1]. Their architecture enables users and applications to provision and manage sensing resources without directly interacting with individual physical sensors. This concept is relevant to OneSensor because the proposed system also represents different sensing capabilities through virtual entities. However, the objective of OneSensor is not cloud-based management of physical sensors. Instead, the ESP32 itself generates the sensor values and provides them as controlled inputs to an external IoT system.

CA4IOT proposed by Perera et al. focuses on context awareness in IoT systems [2]. The system attempts to reduce the complexity of selecting appropriate sensor information by understanding high-level user requirements and automatically configuring sensor filtering, fusion, and reasoning. This work is related to the scenario-based operation of OneSensor because a selected environment can influence the behavior of multiple virtual sensors. However, CA4IOT primarily focuses on context-aware sensor information processing, whereas OneSensor uses environmental scenarios to generate controlled test inputs.

Dynamic sensor configuration was further explored by Perera et al. through the Mobile Sensor Hub approach [3]. Their system provides a mechanism for connecting external sensors and configuring communication between sensor devices and middleware. This is relevant to OneSensor's configurable sensor-to-channel mapping, where the user can determine which virtual sensor should be associated with a particular output channel. The important distinction is that the Mobile Sensor Hub works with physical sensors, while OneSensor generates virtual sensor values.

SensIaaS introduced sensor virtualization within a sensor-cloud infrastructure [4]. The work demonstrates how virtualized sensing resources can be managed as services and made available to applications. Similarly, the Big-Sensor-Cloud Infrastructure proposed by Chatterjee et al. investigates large-scale provisioning of heterogeneous sensing resources as services [5]. These systems demonstrate the scalability and flexibility of cloud-based sensor abstraction. OneSensor instead focuses on a compact embedded platform that can be directly used during IoT hardware development and testing.

Zaidan et al. investigated virtual sensors for high-resolution air-pollution monitoring using artificial intelligence and IoT [6]. Their approach demonstrates how virtual sensing can estimate environmental information using computational models and available data. This work provides motivation for the future extension of OneSensor toward machine-learning-based sensor behavior. In the proposed system, however, the primary purpose of the virtual sensor is not environmental estimation but controlled generation of test inputs for IoT applications.

2.1 Hardware-in-the-Loop and Fault-Injection Literature

A second, largely disjoint body of literature studies HIL testing as a general methodology for embedded and control systems. Abdalla et al. describe a low-cost HIL framework built around a commodity analog/digital I/O card and Simulink, explicitly modeling delay, noise, and saturation so that a controller under test experiences realistic, non-ideal signals rather than an idealized simulation [7]. National Instruments' HIL test-system architecture similarly emphasizes electrical load and sensor emulation together with deterministic real-time I/O as the two pillars of a credible HIL rig [8]. OneSensor adopts the same principle – physically realistic PWM/DAC signals over real wires – but targets a very different price and complexity point, replacing PXI/CompactRIO-class real-time hardware with a single low-cost ESP32 module.

On the fault-injection side, Elgharbawy et al. propose a generic architecture for injecting faults into virtual sensor signals for Advanced Driver Assistance System (ADAS) testing, arguing that fault realism at the signal layer is essential for validating perception pipelines [9]. Magliano et al. present a configurable fault injector that manipulates a CPU's micro-architectural state (registers and RAM) through a JTAG debug interface for reliability assessment of safety-critical embedded software [10], while Skarin et al.'s GOOFI-2 tool provides a generic, campaign-based framework for injecting faults and collecting dependability measurements across heterogeneous target platforms [11]. These frameworks operate primarily at the processor/memory level or at the level of a simulated perception signal; OneSensor is instead positioned at the physical analog/PWM sensor-interface layer that sits between a virtual sensor model and a real target microcontroller's input pins, which is a comparatively underserved point in the fault-

injection design space for low-cost embedded prototyping. Finally, low-cost ESP32 prototyping itself has been documented as an effective platform for IoT experimentation and education because of its integrated Wi-Fi, dual-core processor, and rich peripheral set [12, 13], which supports the choice of the ESP32 as OneSensor's virtual-sensing core.

Table 1 summarizes how OneSensor's positioning differs from the cloud-oriented sensor-virtualization literature and from the HIL/fault-injection literature discussed above.

3. Problem Statement and Research Gap

IoT applications often require multiple sensors during development and testing. Using physical sensors for every required input increases prototype cost and makes the testing process more complicated. In addition, environmental conditions such as fire, gas leakage, flooding, extreme temperature, or abnormal sensor behavior are difficult to reproduce safely and repeatedly. Existing sensor virtualization research has mainly concentrated on virtual representations of physical sensing resources, cloud-based sensor management, context-aware processing, dynamic sensor configuration, and virtual estimation of environmental measurements.

The research gap addressed by OneSensor is the integration of virtual sensor generation and controlled Hardware-in-the-Loop testing on a low-cost embedded platform. The proposed system allows multiple sensor types to be represented by software-generated values on an ESP32 and provides mechanisms for changing their configuration, environmental behavior, and fault conditions. The same platform can therefore be reused for different IoT projects without requiring a separate physical sensor set for every application.

OneSensor also addresses the problem of reproducibility. A particular sensor condition can be generated repeatedly using the same configuration. This is useful for testing applications under abnormal conditions because the developer can reproduce the same sensor behavior without physically recreating the corresponding environmental event.

4. Proposed System

OneSensor is proposed as an ESP32-based virtual multi-sensor platform for IoT development and testing. The central idea is to use the ESP32 as a virtual sensing device rather than connecting a separate physical sensor for every measurement required by an IoT application. The ESP32 generates sensor values according to predefined models and sends the generated values to an external device that represents the system under test.

The initial virtual sensor set consists of temperature, humidity, gas, light, and soil moisture. Each sensor has a configurable range and behavior model. Instead of producing completely random numbers, the system can generate time-dependent values that change gradually under normal conditions. The generated value can subsequently be modified according to the selected environmental scenario or fault condition.

The proposed architecture consists of a web dashboard, configuration manager, scenario engine, virtual sensor models, fault injection engine, channel manager, and output interface. The user interacts with the dashboard through a browser. The selected configuration is processed by the ESP32, which generates the corresponding virtual sensor values and routes them through the configured output channels. Fig. 1 shows this conceptual flow.

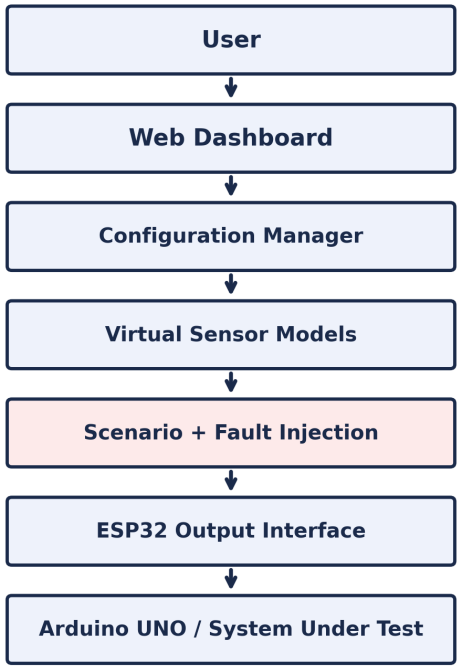


Figure 1. Conceptual architecture of the OneSensor platform.

One of the important features of OneSensor is dynamic sensor-to-channel mapping. The user can select a virtual sensor and assign it to a particular output channel from the dashboard. This provides flexibility because the same ESP32 platform can be configured differently for different IoT projects. For example, a temperature model may be assigned to one channel during one experiment and replaced by a gas model during another experiment without changing the core hardware.

The system also introduces scenario-based simulation. A scenario represents an environmental condition in which multiple sensor values may change together. For example, a fire scenario can produce increasing temperature and gas values, while an agriculture scenario can produce variations in soil moisture, temperature, humidity, and light. This relationship between multiple sensor values provides more meaningful test data than independent random-value generation.

5. System Architecture

The OneSensor architecture is divided into several functional layers, shown in Fig. 2. The first layer is the user interface, implemented as a web dashboard hosted by or connected to the ESP32. The dashboard provides controls for selecting virtual sensors, assigning output channels, choosing scenarios,

Table 1: Comparison of OneSensor with Related Sensor-Virtualization and HIL/Fault-Injection Approaches

Approach	Sensing Model	Signal Delivered to DUT	Fault Injection	Cost / Deployment
Sensor-Cloud Infrastructure [1]	Cloud-managed virtual sensor objects	Data/API values (no electrical signal)	Not addressed	Cloud infrastructure
CA4IOT [2]	Context-aware sensor data processing	Data/API values	Not addressed	Middleware service
Mobile Sensor Hub [3]	Dynamic config. of physical sensors	Data via middleware	Not addressed	Middleware service
SensIaaS / Big-Sensor-Cloud [4, 5]	Sensor-cloud virtualization-as-a-service	Data/API values	Not addressed	Cloud infrastructure
AI Virtual Sensors [6]	ML-estimated environmental measurement	Data values	Not addressed	Cloud / edge compute
Commodity HIL kit [7]	Plant simulation + ADDA card	Analog voltage (8-bit, 2 ch.)	Not addressed	Low-cost ADDA card + PC
Commercial HIL (PXI/SLSC) [8]	Electrical load/sensor emulation	Analog/digital, multi-channel	Signal conditioning faults	High-cost real-time rig
ADAS virtual sensor faults [9]	Simulated perception-sensor signal	Simulated signal (not physical wire)	Rich, sensor-level fault set	Simulation toolchain
OneSensor (proposed)	**ESP32-generated virtual sensor**	**Physical PWM / DAC signal, 12 ch.**	**Dropout, stuck, noise, spike, drift, disconnect, latency**	**Single low-cost microcontroller pair**

configuring fault conditions, and observing generated sensor values.

The second layer is the configuration manager. It stores the current sensor selection and channel mapping. Each output channel can be associated with a particular virtual sensor according to the requirements of the IoT application being tested. This allows the same physical OneSensor device to support different application configurations.

The third layer contains the virtual sensor models. Each model is responsible for generating values corresponding to a specific sensor type. The models can use predefined ranges, gradual changes, relationships between variables, and scenario-specific behavior. The generated values are passed to the fault injection layer before being delivered to the output interface. As summarized in Table 2, the current prototype implements twelve concurrently addressable channels, each described by a configuration record giving its output GPIO, signal type, dynamic range, default value, and calibration factors.

The fourth layer is the scenario and fault injection layer. Scenarios modify sensor behavior according to predefined environmental conditions. Fault injection intentionally introduces abnormal behavior such as noisy data, stuck values, sudden spikes, sensor offline conditions, or delayed transmission.

The final layer is the output interface. It transfers the generated values from the ESP32 to the external system under test. In the prototype, an Arduino UNO can be used as the receiving device. The Arduino processes the incoming sensor values as inputs for the application being tested; a companion decoder library recovers the pulse duty cycle (for PWM channels) or the sampled analog voltage (for DAC channels) and converts it back into the original engineering unit through the inverse of the linear mapping applied on the ESP32 side.

6. Fault Injection and Testing

Fault injection is included to evaluate how an IoT application responds to abnormal sensor conditions. The system can intentionally introduce conditions such as sensor offline behavior, noisy readings, stuck values, sudden spikes, gradual drift, and delayed transmission. Because these faults are software-controlled, the same condition can be reproduced multiple times during testing.

A normal sensor value can be represented as

$$S(t) = B + T(t) + E(t) + N(t), \tag{1}$$

where B is the baseline value, $T(t)$ represents the time-dependent component, $E(t)$ represents the environmental scenario, and $N(t)$ represents controlled variation.

The fault injection layer transforms the normal value according to the selected fault condition. For example, a spike fault can temporarily increase the value, while a stuck-sensor fault can maintain a previously generated value for a specified period. Table 3 summarizes the seven fault classes implemented in the current prototype and the transformation each applies to the incoming value before it reaches the output interface, shown schematically in Fig. 3.

The testing process follows a Hardware-in-the-Loop concept. The ESP32 generates the virtual sensor input while the Arduino UNO executes the application logic normally used with sensor data. The developer can therefore evaluate the behavior of the external application under normal, environmental, and faulty conditions.

Table 4 positions OneSensor's fault-injection layer relative to the fault-injection frameworks discussed in Section 2.1. Unlike frameworks that manipulate processor registers or memory [10] or a generic dependability-campaign target [11], and

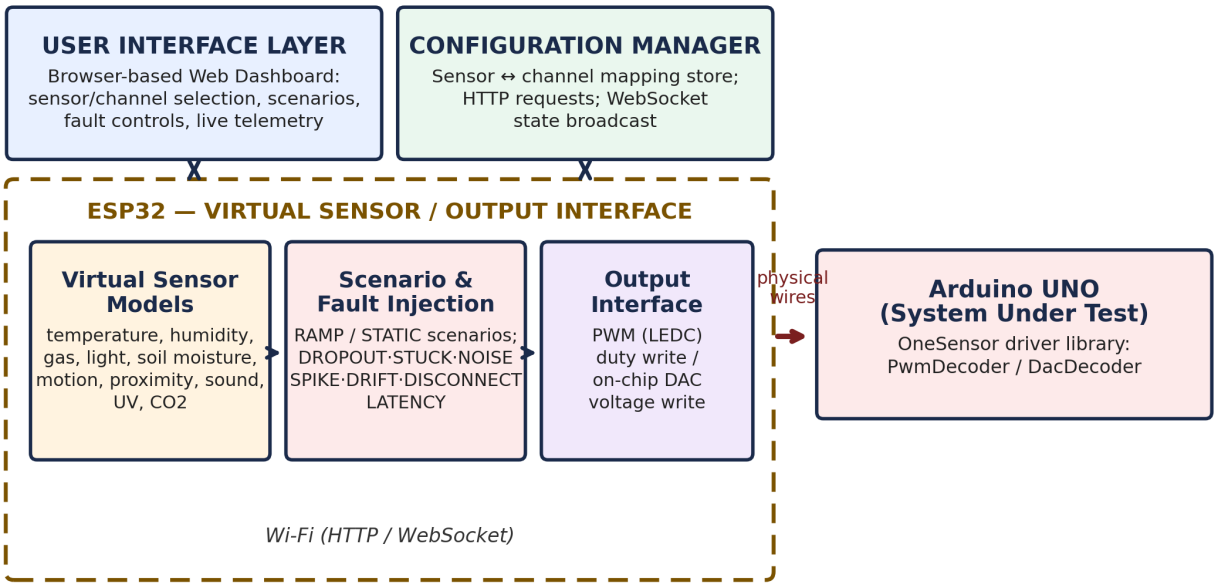


Figure 2. Layered system architecture of OneSensor, showing the dashboard and configuration layers, the virtual-sensor, scenario, fault, and output layers hosted inside the ESP32, and the physically wired Arduino UNO target.

unlike ADAS fault injection that operates on a simulated perception signal [9], OneSensor injects faults directly into the physical analog/PWM signal delivered to a real target microcontroller, which exercises the target's actual input-decoding and signal-conditioning code path rather than only its application logic.

7. Implementation

The prototype is implemented using an ESP32 development board and an Arduino UNO. The ESP32 is the primary OneSensor device and is responsible for virtual sensor generation and configuration. The Arduino UNO is treated as the external system under test. The initial implementation establishes the communication path between the two devices before progressively adding the complete dashboard and simulation features.

In the initial stage, the ESP32 generates a virtual sensor value and provides an equivalent electrical output that can be read by the Arduino. This approach validates the basic concept that software-generated sensor information can be supplied to another microcontroller as an input. The signal path is therefore ESP32 virtual sensor generation, output interface, Arduino input, and application processing.

The ESP32 provides Wi-Fi connectivity that can be used for communication with the browser-based dashboard. HTTP can be used for dashboard requests and configuration, while WebSocket communication can provide real-time updates of generated sensor values. MQTT can optionally be supported for external IoT integration. Table 5 summarizes the software stack used across the two ends of the current prototype.

Fig. 4 and Fig. 5 indicate the intended layout of the two primary dashboard views once the browser-based front end is finalized: a live telemetry view showing the twelve virtual channels, and a configuration view for sensor-to-channel mapping and fault arming.

The system is designed in a modular manner so that the virtual sensor generation layer is independent of the dashboard and output layer. This makes it possible to add additional virtual sensors without redesigning the complete system. It also provides a foundation for future machine-learning-based sensor models.

8. IoT Technology and Communication

The ESP32 is selected as the main controller because it provides sufficient processing capability, Wi-Fi connectivity, multiple GPIO interfaces, and a low-cost development platform [12, 13]. Wi-Fi enables the OneSensor device to communicate with a browser-based dashboard without requiring additional networking hardware.

HTTP is suitable for configuration and dashboard requests because it provides a simple communication mechanism between the browser and the ESP32. WebSocket can be used when continuous real-time sensor updates are required because it allows persistent communication between the dashboard and the device. MQTT can be considered for future integration with external IoT platforms where a publish-subscribe communication model is required.

Table 2: Virtual Sensor Channel Configuration in the Current Prototype

Ch.	Virtual Sensor	Signal Type	Output GPIO	Range	Default	Encoding
0	Temperature	DAC	25	0–50 °C	25 °C	8-bit DAC
1	Humidity	DAC	26	0–100 %	50 %	8-bit DAC
2	Gas	PWM	18	0–1000 ppm	300 ppm	10-bit, 500 Hz
3	Light	PWM	19	0–1000 lux	500 lux	10-bit, 500 Hz
4	Soil Moisture	PWM	21	0–100 %	50 %	10-bit, 500 Hz
5	Motion X	PWM	22	−2–+2 g	0 g	10-bit, 500 Hz
6	Motion Y	PWM	23	−2–+2 g	0 g	10-bit, 500 Hz
7	Motion Z	PWM	27	−2–+2 g	1 g	10-bit, 500 Hz
8	Proximity	PWM	32	2–400 cm	50 cm	10-bit, 500 Hz
9	Sound Level	PWM	33	30–120 dB	40 dB	10-bit, 500 Hz
10	UV Index	PWM	14	0–11 idx	2 idx	10-bit, 500 Hz
11	CO_2	PWM	12	400–5000 ppm	420 ppm	10-bit, 500 Hz

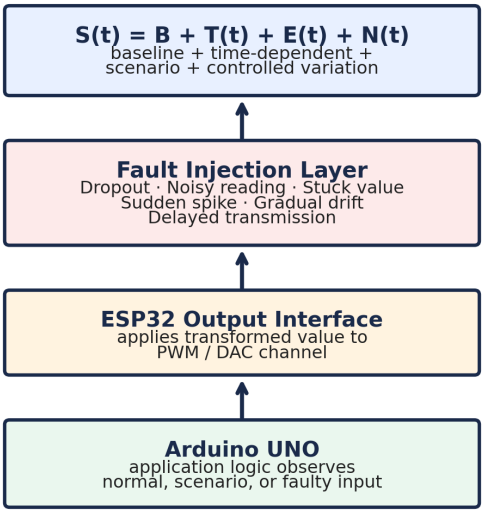


Figure 3. Signal path from the baseline sensor model, through scenario composition and fault injection, to the Arduino UNO under test.

The Arduino UNO acts as the system under test. It receives the generated sensor input and executes the application logic. This separation between the OneSensor device and the application controller makes it possible to test different IoT applications using the same virtual sensor platform.

9. Expected Results and Discussion

The expected result of the OneSensor prototype is a functional ESP32-based device capable of generating multiple virtual sensor values and providing them to an external IoT controller. The user should be able to select sensor types, configure their output channels, select predefined environmental scenarios, and introduce controlled sensor faults.

The system is expected to reduce the number of physical sensors required during the initial development and testing stages. It also provides a repeatable method for testing abnormal conditions. For example, a developer can repeatedly apply a sudden gas-value spike or sensor-offline condition and observe how the application responds.

The proposed system is not intended to completely replace physical sensors during final deployment. Physical sensors are still required when testing actual electrical characteristics, calibration behavior, hardware noise, and real environmental measurements. OneSensor primarily addresses software and application-level testing where controlled sensor inputs are sufficient.

The prototype also provides a foundation for automated

Table 3: Fault Transformation Summary

Fault	Transformation	Behavior
Dropout	$y(t)=y(t_{\text{last}})$	Freezes at last valid output
Stuck	$y(t)=M$	Clamps to constant M
Noise	$y(t)=x(t)+\mathcal{N}(0,M^2)$	Zero-mean Gaussian jitter
Spike	$y(t)=x(t)+\Delta_{\text{spike}}$	Transient large excursion
Drift	$y(t)=x(t)+M\cdot\frac{t-t_0}{1000}$	Accumulating linear offset
Disconnect	$y(t)=\text{NaN}$	Mutes electrical output
Latency	$y(t)=x(t-\Delta t)$	Ring-buffer delayed replay

Table 4: Fault-Injection Target Layer Comparison

Framework	Fault Target Layer	Physical Signal?
GOOFI-2 [11]	Processor/memory (campaign-based)	No
Magliano et al. [10]	CPU registers / RAM via JTAG	No
ADAS sensor faults [9]	Simulated perception signal	No
OneSensor	**Physical PWM/DAC sensor signal**	**Yes**

Table 5: Implementation Stack Summary

Tier	Technology
ESP32 firmware	Arduino core on ESP32, asynchronous HTTP/WebSocket server, LEDC PWM and on-chip DAC drivers
Dashboard client	Browser-based single-page application communicating over HTTP and WebSocket
Target device	Arduino UNO (ATmega328P) with a pulse-width/analog decoder library

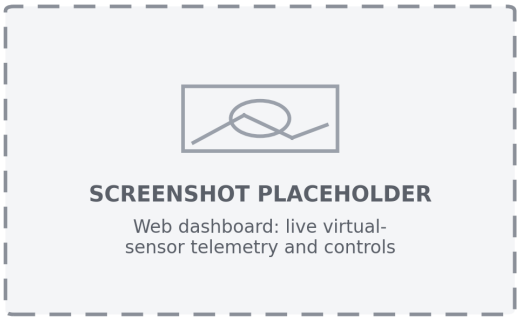


Figure 4. Placeholder for the live-telemetry dashboard screenshot.

testing. Multiple scenarios and faults can be executed sequentially, allowing developers to observe the application's response across different operating conditions. Quantities such as end-to-end latency, PWM phase jitter under simultaneous Wi-Fi activity, and Arduino-side decoder CPU overhead are architectural properties that can be derived from the channel and timing configuration in Table 2, but they require empirical measurement on physical hardware and are left for future evaluation rather than claimed here.

10. Future Scope

The current architecture provides several possibilities for future development. Machine-learning-based virtual sensor models can be introduced by training models using real sensor datasets. Instead of relying only on mathematical equations, the system could learn temporal patterns, correlations between sensors, and realistic variations from recorded measurements. This can improve the realism of generated sensor data.

Another direction is protocol-level sensor emulation. Future versions can investigate emulation of interfaces such as I2C, SPI, UART, and OneWire communication. This would allow OneSensor to test applications that depend on existing sensor libraries without requiring the actual sensor hardware.

The dashboard can also be extended with automated test sequences. A developer could define a sequence containing normal operation, scenario changes, fault injection, recovery, and data logging. The platform could execute the sequence automatically and generate a report of the observed system behavior.

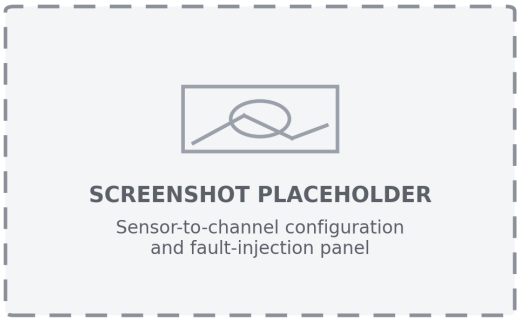


Figure 5. Placeholder for the sensor-to-channel configuration and fault-injection panel screenshot.

Additional future work includes cloud integration, mobile monitoring, more virtual sensor models, dataset-based scenario generation, a plugin architecture through which developers can add new sensor models without modifying the core system, and a systematic empirical benchmarking study – along the lines of the HIL evaluation methodology reported for commodity ADDA-card rigs [7] – to quantify end-to-end latency, jitter, and decoder overhead on physical hardware.

11. Conclusion

OneSensor presents a low-cost ESP32-based approach for virtualizing multiple sensor inputs for IoT development and testing. The proposed system uses the ESP32 as the main virtual sensor device and an Arduino UNO or another controller as the system under test. By generating configurable sensor values, the platform can reduce dependence on multiple physical sensors during the development stage.

The combination of sensor-to-channel mapping, environmental scenario simulation, fault injection, and browser-based configuration provides a controlled environment for testing IoT applications. Unlike approaches that primarily focus on cloud-based sensor management, OneSensor focuses on providing virtual sensor inputs directly to an embedded IoT system, and unlike fault-injection frameworks that target processor state or simulated signals, OneSensor injects faults directly into the physical sensor-interface layer seen by the target device.

The current prototype establishes the fundamental ESP32-to-Arduino testing concept across twelve virtual sensor channels, while future development will extend the platform toward more accurate sensor protocol emulation, machine-learning-based sensor behavior, automated test sequences, and cloud connectivity. The proposed platform therefore provides a practical foundation for low-cost, reusable, and repeatable IoT development and testing.

Acknowledgment

The authors would like to thank their project guide and the Department of Computer Engineering for their guidance and support during the development of the OneSensor project.

References

[1] M. Yuriyama and T. Kushida, "Sensor-Cloud Infrastructure: Physical Sensor Management with Virtualized Sensors on Cloud Computing," in *Proc. 13th Int. Conf. Network-Based Information Systems*, 2010, pp. 1–8.

[2] C. Perera, A. Zaslavsky, P. Christen, and D. Georgakopoulos, "CA4IOT: Context Awareness for Internet of Things," in *Proc. IEEE Green Computing and Communications and IEEE Internet of Things and IEEE Cyber, Physical and Social Computing*, 2012, pp. 775–782.

[3] C. Perera, P. Jayaraman, A. Zaslavsky, P. Christen, and D. Georgakopoulos, "Dynamic Configuration of Sensors Using Mobile Sensor Hub in Internet of Things Paradigm," in *Proc. IEEE Int. Conf. Intelligent Sensors, Sensor Networks and Information Processing*, 2013, pp. 473–478.

[4] S. Bose and N. Mukherjee, "SensIaaS: A Sensor-Cloud Infrastructure with Sensor Virtualization," in *Proc. IEEE 3rd Int. Conf. Cyber Security and Cloud Computing*, 2016, pp. 232–239.

[5] S. Chatterjee et al., "Big-Sensor-Cloud Infrastructure: A Holistic Prototype for Provisioning Sensors-as-a-Service," *IEEE Trans. Cloud Computing*, 2021.

[6] M. A. Zaidan et al., "Virtual Sensors: Toward High-Resolution Air Pollution Monitoring using AI and IoT," *IEEE Internet of Things Magazine*, vol. 6, no. 1, pp. 76–81, 2023.

[7] O. M. Abdalla, S. A. Hammad, and A. H. Yousef, "A Framework for Real-Time Hardware-in-the-Loop Simulation for Control Design," *arXiv preprint arXiv:1410.1342*, 2014.

[8] National Instruments, "The Hardware-in-the-Loop (HIL) Test System Architecture," NI White Paper. [Online]. Available: https://www.ni.com/en/solutions/transportation/hardware-in-the-loop/hardware-in-the-loop–hil–test-system-architectures.html

[9] M. Elgharbawy, A. Schwarzhaupt, G. Scheike, M. Frey, and F. Gauterin, "A Generic Architecture of ADAS Sensor Fault Injection for Virtual Tests," in *Proc. IEEE/ACS 13th Int. Conf. Computer Systems and Applications (AICCSA)*, 2016.

[10] E. Magliano, A. Savino, and S. Di Carlo, "Real-Time Embedded System Fault Injector Framework for Micro-Architectural State Based Reliability Assessment," *Journal of Electronic Testing*, vol. 41, no. 2, pp. 193–208, 2025.

[11] D. Skarin, R. Barbosa, and J. Karlsson, "GOOFI-2: A Tool for Experimental Dependability Assessment," in *Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, 2010, pp. 557–562.

[12] D. Hercog, T. Lerher, M. Truntič, and O. Težak, "Design and Implementation of ESP32-Based IoT Devices," *Sensors*, vol. 23, no. 15, p. 6739, 2023.

[13] A. Maier, A. Sharp, and Y. Vagapov, "Comparative Analysis and Practical Implementation of the ESP32 Microcontroller Module for the Internet of Things," in *Proc. 7th IEEE Int. Conf. Internet Technologies and Applications (ITA-17)*, Wrexham, U.K., 2017.